

Forecasting Sticker Sales — Project Report

Kaggle Playground Series S5E1 — playground-series-s5e1

1. Problem Statement

The task is a time-series regression problem: predicting the number of stickers sold (`num_sold`) for combinations of country, store and product over time, based on the Kaggle Playground Series S5E1 competition ("Forecasting Sticker Sales").

Data

- Source: Kaggle competition `train.csv` / `test.csv` (`playground-series-s5e1`).
- 90 unique (country, store, product) time series, each with a complete, continuous set of dates — no missing dates.
- Train period: 2010–2016. Kaggle test period: 2017–2019 (a 3-year forecast horizon).
- 8,871 rows (~3.85%) of train have a missing target `num_sold`, concentrated mostly in 'Holographic Goose' for Canada and Kenya.
- Missing targets are dropped for supervised training/evaluation (`drop_missing_target()`); they are not imputed with mean/median, and this function is not applied to the test set.

Evaluation Metric

Mean Absolute Percentage Error (MAPE), computed with the shared `mape()` function in `src/metrics.py`, so that all baseline and model results are directly comparable.

Validation Scheme

Because this is a forecasting task, a random train/test split would leak future information into training, so a purely temporal (expanding-window) validation scheme is used instead:

- Fold 1: train 2010–2013 → validate 2014
- Fold 2: train 2010–2014 → validate 2015
- Fold 3: train 2010–2015 → validate 2016

A long-horizon backtest is used in addition, to mimic the real Kaggle test horizon (3 years ahead):

- train 2010–2013 → validate 2014–2016 (3-year horizon, same length as the real 2017–2019 test period)

2. Baselines

Three baseline models were implemented and evaluated with the shared validation scheme and `mape()`:

- `group_median` — predicts the median `num_sold` for each (country, store, product) group.
- `seasonal_naive_last_year` — uses the value from the same calendar date one year earlier; falls back to the group median when that value is unavailable.
- `ridge_calendar_one_hot` — Ridge regression on calendar features plus one-hot encoded country/store/product.

Model	Expanding-window MAPE (%)	Long-horizon MAPE (%)
group_median	16.32	17.46
seasonal_naive_last_year (best baseline)	13.27	17.02
ridge_calendar_one_hot	17.86	23.91

Expanding-window MAPE is the mean over the three folds (2014: 12.55 / 11.23 / 13.28, 2015: 18.25 / 15.90 / 22.08, 2016: 18.16 / 12.69 / 18.22 for group_median / seasonal_naive_last_year / ridge_calendar_one_hot respectively — see results/metrics.csv for the exact per-fold values).

seasonal_naive_last_year was selected as the strongest baseline and the reference point for the proposed model, since it already captures the dominant yearly seasonality in the data without any learned parameters.

3. Proposed Model

Version 1 — catboost_raw, v1 (src/model.py + notebooks/03_model.ipynb)

A CatBoostRegressor trained directly on the raw num_sold target.

- Features: 15 shared calendar features (year, month, day, day_of_week, day_of_year, week_of_year, quarter, is_weekend, time_idx and weekly/yearly cyclical sin-cos features) plus the categorical features country, store, product.
- Parameters: 500 iterations, depth=6, learning_rate=0.05, loss_function=RMSE, seed=42, CPU, 4 threads, has_time=True.
- No hyperparameter search and no early stopping in v1.
- Negative predictions are clipped to zero.

Split	MAPE (%)
2014 (fold 1)	15.92
2015 (fold 2)	23.76
2016 (fold 3)	15.06
Mean expanding-window	18.24
Long-horizon (2014–2016)	22.52

v1 is worse than the seasonal_naive_last_year baseline (18.24% vs 13.27% expanding-window; 22.52% vs 17.02% long-horizon), indicating that, in its raw form, CatBoost was not yet using the available information more effectively than simply repeating last year's seasonal pattern.

4. Model Improvements

Version 2 — catboost_log, log1p target (v2)

The only change from v1 is training on log1p(num_sold) instead of the raw target, and inverse-transforming predictions with expm1 before evaluation. All features, splits and CatBoost hyperparameters are unchanged from v1.

Model version	Expanding-window MAPE (%)	Long-horizon MAPE (%)
v1 — raw target	18.24	22.52
v2 — log1p target	9.17	14.94

The log-target transform gave the single largest improvement in the project, and made the model surpass both v1 and the seasonal_naive_last_year baseline. This is consistent with num_sold being right-skewed / multiplicative in nature across country-store-product combinations of very different scale — a log target lets the model learn relative rather than absolute errors, which matches how MAPE itself is computed.

Version 3 — catboost_log_tuned, hyperparameter tuning (v3)

Starting from v2 (log-target kept), a small hyperparameter search was run over depth, learning_rate, iterations and l2_leaf_reg, using only the expanding-window folds for selection.

Best configuration found: 500 iterations, depth=8, learning_rate=0.05, l2_leaf_reg=3.

Model version	Expanding-window MAPE (%)	Long-horizon MAPE (%)
v1 — raw target	18.24	22.52
v2 — log1p target	9.17	14.94
v3 — log1p target + tuned depth/l2	9.04	14.60

Tuning gave a further, smaller improvement on top of the log-target change (9.17% → 9.04% expanding-window; 14.94% → 14.60% long-horizon), confirming that the target transform, not the hyperparameters, was the main driver of the gain.

5. Results

Summary table — all models

Model	Expanding-window MAPE (%)	Long-horizon MAPE (%)
group_median (v1)	16.32	17.46
seasonal_naive_last_year (v1, best baseline)	13.27	17.02
ridge_calendar_one_hot (v1)	17.86	23.91
catboost_raw (v1)	18.24	22.52
catboost_log (v2, log1p target)	9.17	14.94
catboost_log_tuned (v3, log1p + tuned depth/l2)	9.04	14.60

Model/version names match the 'model' and 'version' columns in results/metrics.csv exactly, so every number here can be traced back to a specific row.

Final model (v3) improves on the strongest baseline by roughly 4.2 percentage points on expanding-window MAPE (13.27% → 9.04%) and 2.4 points on the long-horizon backtest (17.02% → 14.60%), i.e. a ~32% and ~14% relative error reduction respectively.

Error analysis

Produced by notebooks/04_evaluation.ipynb, which trains the final v3 configuration (depth=8, l2_leaf_reg=3, log1p target) on the long-horizon split (train 2010-2013, validate 2014-2016) and compares it against seasonal_naive_last_year on the same validation rows (results/error_analysis_by_*.csv, results/error_analysis_worst_series.png).

MAPE by year (long-horizon window)

Year	v3 MAPE (%)	seasonal_naive_last_year MAPE (%)	v3 advantage (p.p.)
2014	7.06	11.23	4.17
2015	17.42	18.95	1.53
2016	19.35	20.90	1.55

Error grows for both models as the forecast horizon lengthens, which is expected for a 3-year-ahead backtest. More notably, v3's advantage over the baseline is concentrated in the first forecast year: it beats the baseline by 4.17 p.p. in 2014, but the gap narrows to about 1.5 p.p. by 2015-2016, i.e. most of the model's edge over a purely seasonal repeat comes from the near horizon, and it converges toward the baseline further out.

MAPE by country

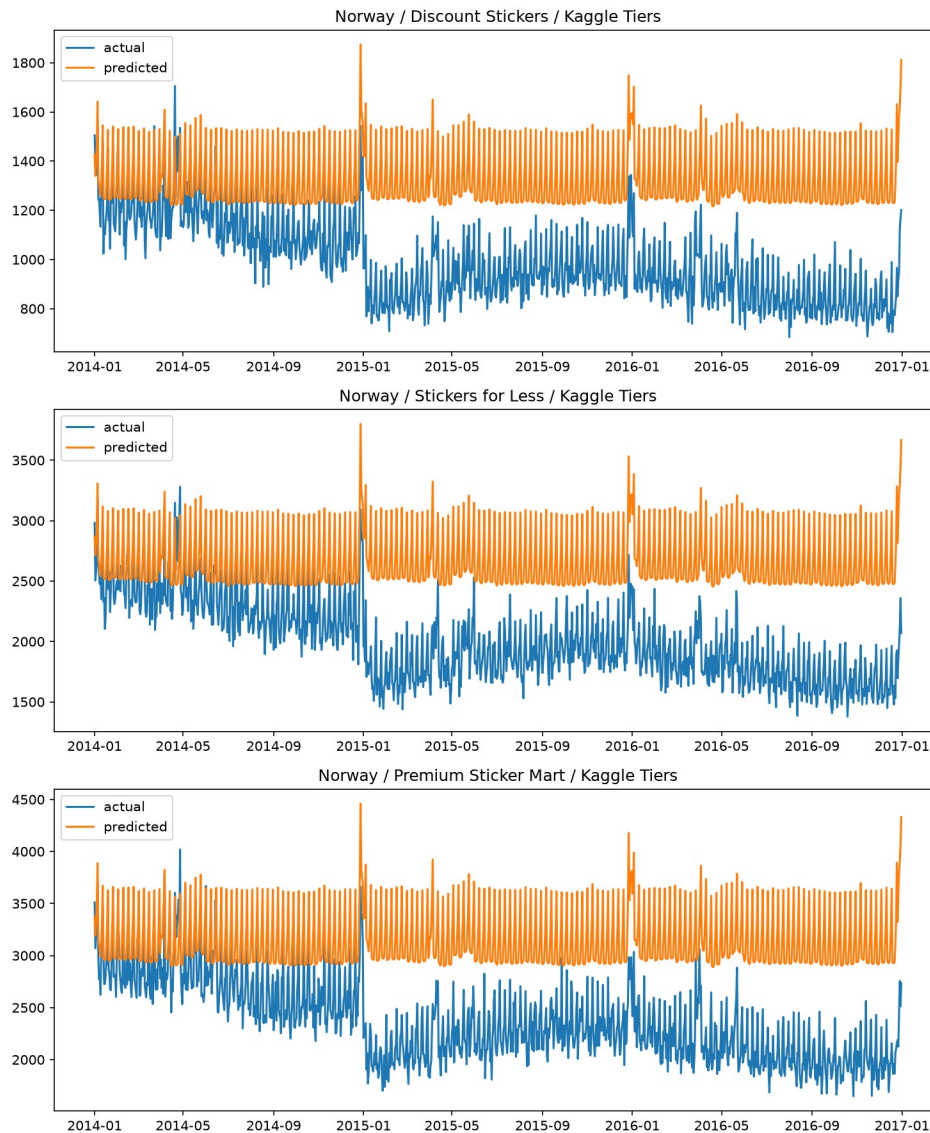
Country	v3 MAPE (%)	baseline MAPE (%)	gap (baseline – v3)
Singapore	5.63	11.69	+6.06
Kenya	11.04	19.98	+8.94
Finland	11.87	13.35	+1.48
Italy	11.88	15.43	+3.55
Canada	16.30	15.77	-0.53
Norway	30.81	25.97	-4.84

Norway is by far the hardest country for the model (30.81% MAPE) and is the only country where v3 is actually worse than the seasonal-naive baseline (-4.84 p.p.). Canada is close to a tie. On the other four countries the model clearly beats the baseline, most dramatically on Kenya (+8.94 p.p.) and Singapore (+6.06 p.p.).

MAPE by store and by product

By store, v3 beats the baseline by a fairly uniform 2.2-2.6 p.p. across all three stores (Discount Stickers, Premium Sticker Mart, Stickers for Less), so store is not where the model's weakness concentrates. By product, the model is roughly tied with the baseline on Kaggle Tiers (17.33% vs 17.16%, a slight loss of 0.17 p.p.), and clearly better on Kerneler Dark Mode (+4.99 p.p.), Kerneler (+3.89 p.p.), Holographic Goose (+2.36 p.p., despite this being the product with the missing-target rows during training) and Kaggle (+1.01 p.p.).

Worst series: Norway, all three stores, 'Kaggle Tiers'



All three worst-performing series by mean absolute percentage error are Norway / Kaggle Tiers, across all three stores. The plots show a clear, consistent pattern: actual sales (blue) step down sharply right at the start of 2015 and stay at that lower level through 2016, while the model's predictions (orange) keep following roughly the same seasonal level and amplitude as in 2014. Because the model's features are purely calendar-based (date-derived) plus static country/store/product identity, it has no mechanism to pick up a level shift in a specific country's demand that isn't tied to the calendar — it can only repeat the pattern it learned from earlier years for that (country, store, product) combination. This produces a persistent over-prediction bias for Norway from 2015 onward, and explains both why Norway has by far the worst country-level MAPE and why the model loses to the simpler seasonal-naïve baseline specifically there (the baseline, by contrast, reacts one year later once the lower 2015 values become 'last year' for 2016).

Overall, the error analysis suggests the model's remaining weakness is not random noise but a structural one: it cannot detect country-specific demand-level shifts that aren't encoded in the calendar features, which matches the Limitations discussed in Section 7 (no lag/rolling or trend features).

6. Reproducibility

Environment

- Python 3.10+ (development/testing done with Python 3.14.4, pandas 3.0.5, NumPy 2.5.3, CatBoost 1.2.10 — versions are not pinned in requirements.txt).
- Install dependencies: `pip install -r requirements.txt`
- Libraries used: pandas, numpy, matplotlib, scikit-learn, catboost, plus jupyter/ipykernel to run the notebooks.

Data access

- Download train.csv / test.csv (and sample_submission.csv) from <https://www.kaggle.com/competitions/playground-series-s5e1/data>
- Place the files in data/ at the repository root — data/*.csv is git-ignored, so this step is required after every fresh clone.

Step-by-step to reproduce all reported results

1. git clone <https://github.com/ariagaf/sticker-sales-forecasting.git> && cd sticker-sales-forecasting
2. python -m venv .venv && source .venv/bin/activate (Windows: .venv\Scripts\activate)
3. pip install -r requirements.txt
4. Place train.csv / test.csv in data/
5. Run notebooks/01_eda.ipynb — EDA and sanity checks, no metrics produced.
6. Run: python -m src.baselines (equivalently, run notebooks/02_baselines.ipynb) → writes the group_median / seasonal_naive_last_year / ridge_calendar_one_hot rows to results/metrics.csv.
7. Run notebooks/03_model.ipynb top to bottom → writes catboost_raw (v1), catboost_log (v2) and catboost_log_tuned (v3) rows to results/metrics.csv. The v3 hyperparameter search reuses src/model.py's fit_model()/predict_model() with an overriding params dict {iterations:500, depth:8, learning_rate:0.05, l2_leaf_reg:3}; src/model.py itself was not modified for v3.
8. Run notebooks/04_evaluation.ipynb (this report's Stage-6 addition) → writes results/error_analysis_by_{country,store,product,year}.csv and results/error_analysis_worst_series.png, used in Section 5.
9. Compare results/metrics.csv against the tables in Sections 2-5 of this report — they should match to at least 2 decimal places.

TODO (fill after running the notebook): Run this checklist on a clean checkout / fresh environment before submission and record here: date of the check, machine/OS, and confirmation that all metrics reproduced exactly (03_model.ipynb's own reproducibility note states a fresh-kernel run reproduced all four v1/v2 scores without duplicating CSV rows; confirm the same for v3 and for your own 04_evaluation.ipynb run).

7. Conclusion

The strongest simple baseline, seasonal_naive_last_year, already captures most of the yearly seasonality in the data (13.27% expanding-window MAPE / 17.02% long-horizon MAPE) and is a non-trivial bar to beat. A raw-target CatBoost model (v1) initially underperformed this baseline. Switching the target to $\log_{1p}(\text{num_sold})$ (v2) was the single most impactful change in the project, cutting expanding-window MAPE roughly in half (18.24% → 9.17%) and beating the baseline by a wide margin. A subsequent hyperparameter search (v3) gave a further, smaller improvement (9.17% → 9.04% expanding-window; 14.94% → 14.60% long-horizon).

Limitations:

- Only calendar and categorical features are used; no lag/rolling features were added, specifically to avoid leakage, so the model may be leaving useful signal on the table.
- Hyperparameter tuning was limited to a small grid over 4 parameters and selected only on expanding-window folds, not the long-horizon backtest.
- No ensembling or model averaging was attempted.
- Missing-target rows (Holographic Goose, Canada/Kenya) are dropped rather than modeled explicitly, which could bias those specific series.

Suggested future work:

- Add carefully-lagged/rolling features that only use information available at prediction time (e.g. same-weekday-last-N-years averages).
- Widen the hyperparameter search and validate the final configuration on the long-horizon backtest directly, since that is closer to the real Kaggle test horizon.
- Investigate per-segment models or sample weighting for the low-volume / missing-target series.